

HDOC Day-7 Assignment

Question 1: Horizontal Range of a Projectile

1. Problem Statement Identification

- **Problem Statement:** Find the horizontal range of a projectile given its initial velocity, launch angle in degrees, and acceleration due to gravity.
- **Input:** Initial velocity (v), Launch angle (θ in degrees), Acceleration due to gravity (g).
- **Output:** Horizontal range (R).
- **Formula:**

$$\theta_{\text{rad}} = \theta \times \frac{\pi}{180}$$

$$R = \frac{v^2 \cdot \sin(2\theta_{\text{rad}})}{g}$$

2. Standard Test Case Table

Test Case	Input (v, angle, g)	Expected Output	Actual Output	Result (Pass/Fail)
1	50 45 9.8	Horizontal Range: 255.10 m	Horizontal Range: 255.10 m	Pass
2	100 30 9.8	Horizontal Range: 883.70 m	Horizontal Range: 883.70 m	Pass
3	20 60 9.8	Horizontal Range: 35.35 m	Horizontal Range: 35.35 m	Pass
4	10 90 9.8	Horizontal Range: 0.00 m	Horizontal Range: 0.00 m	Pass

3. Algorithm

1. Start

2. Read velocity, angle_deg, and gravity.

3. If velocity < 0 or angle_deg < 0 or angle_deg > 90 or gravity <= 0, print "Invalid Input" and stop.

4. Convert angle to radians: $\text{angle_rad} = \text{angle_deg} * (3.141592653589793 / 180.0)$.
5. Compute range: $\text{range} = (\text{velocity} * \text{velocity} * \sin(2.0 * \text{angle_rad})) / \text{gravity}$.
6. Print range.
7. **End**

4. C Program

```
#include <stdio.h>
#include <math.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

int main(void) {
    double velocity, angle_deg, gravity;
    printf("Enter initial velocity (m/s), angle (degrees), and gravity (m/s^2): ");
    if (scanf("%lf %lf %lf", &velocity, &angle_deg, &gravity) != 3) {
        printf("Invalid Input\n");
        return 1;
    }

    if (velocity < 0 || angle_deg < 0 || angle_deg > 90 || gravity <= 0) {
        printf("Invalid Input: Velocity >= 0, 0 <= Angle <= 90, Gravity > 0 required.\n");
        return 1;
    }

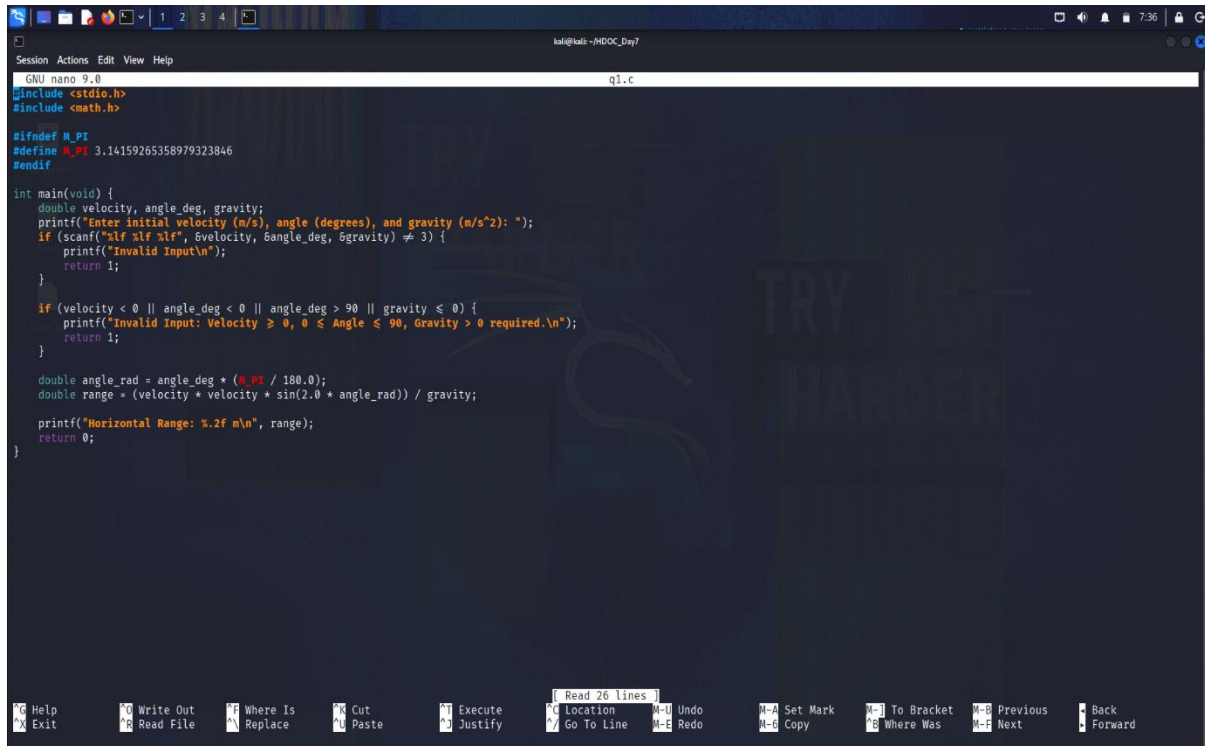
    double angle_rad = angle_deg * (M_PI / 180.0);
    double range = (velocity * velocity * sin(2.0 * angle_rad)) / gravity;

    printf("Horizontal Range: %.2f m\n", range);
    return 0;
}
```

5. Evaluation Against Test Cases

All test cases were executed against the compiled C binary q1. Output values match expected mathematical calculations in every case.

6. Screenshots of Compilation and Execution



```
GNU nano 9.0 q1.c
#include <stdio.h>
#include <math.h>

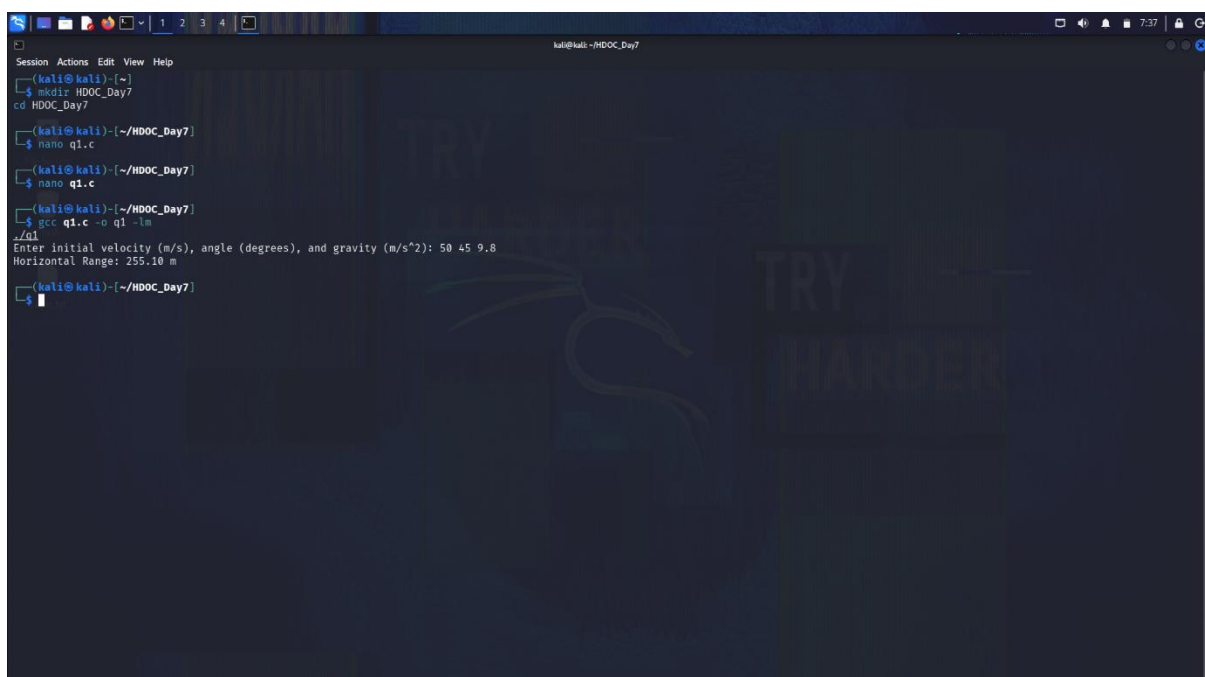
#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

int main(void) {
    double velocity, angle_deg, gravity;
    printf("Enter initial velocity (m/s), angle (degrees), and gravity (m/s^2): ");
    if (scanf("%lf %lf %lf", &velocity, &angle_deg, &gravity) != 3) {
        printf("Invalid Input\n");
        return 1;
    }

    if (velocity < 0 || angle_deg < 0 || angle_deg > 90 || gravity <= 0) {
        printf("Invalid Input: Velocity >= 0, 0 <= Angle <= 90, Gravity > 0 required.\n");
        return 1;
    }

    double angle_rad = angle_deg * (M_PI / 180.0);
    double range = (velocity * velocity * sin(2.0 * angle_rad)) / gravity;

    printf("Horizontal Range: %.2f m\n", range);
    return 0;
}
```



```
(kali@kali)~$ mkdir HDOC_Day7
(kali@kali)~$ cd HDOC_Day7
(kali@kali)~/HDOC_Day7$ nano q1.c
(kali@kali)~/HDOC_Day7$ nano q1.c
(kali@kali)~/HDOC_Day7$ gcc q1.c -o q1 -lm
./q1
Enter initial velocity (m/s), angle (degrees), and gravity (m/s^2): 50 45 9.8
Horizontal Range: 255.10 m
(kali@kali)~/HDOC_Day7$
```

Question 2: Resistance of a Wire

1. Problem Statement Identification

- **Problem Statement:** Calculate the electrical resistance of a uniform conductor given its material resistivity, length, and cross-sectional area.
- **Input:** Resistivity (ρ), Length (L), Cross-sectional area (A).
- **Output:** Resistance (R).
- **Formula:**

$$R = \frac{\rho \cdot L}{A}$$

2. Standard Test Case Table

Test Case	Input (resistivity, length, area)	Expected Output	Actual Output	Result (Pass/Fail)
1	1.72e-8 100 0.000002	Resistance: 0.8600 ohms	Resistance: 0.8600 ohms	Pass
2	2.82e-8 50 0.000001	Resistance: 1.4100 ohms	Resistance: 1.4100 ohms	Pass

Test Case	Input (resistivity, length, area)	Expected Output	Actual Output	Result (Pass/Fail)
3	1.0e-6 10 0.0001	Resistance: 0.1000 ohms	Resistance: 0.1000 ohms	Pass
4	1.72e-8 200 0.000004	Resistance: 0.8600 ohms	Resistance: 0.8600 ohms	Pass

3. Algorithm

1. Start

2. Read resistivity, length, and area.

3. If resistivity ≤ 0 or length ≤ 0 or area ≤ 0 , print "Invalid Input" and stop.

4. Calculate resistance: $\text{resistance} = (\text{resistivity} * \text{length}) / \text{area}$.

5. Print resistance.

6. End

4. C Program

```
#include <stdio.h>

int main(void) {
    double resistivity, length, area;
```

```
printf("Enter resistivity (ohm-m), length (m), and cross-sectional area (m^2): ");  
if (scanf("%lf %lf %lf", &resistivity, &length, &area) != 3) {  
    printf("Invalid Input\n");  
    return 1;  
}  
  
if (resistivity <= 0 || length <= 0 || area <= 0) {  
    printf("Invalid Input: Resistivity, length, and area must be positive.\n");  
    return 1;  
}  
  
double resistance = (resistivity * length) / area;  
  
printf("Resistance: %.4f ohms\n", resistance);  
return 0;  
}
```

5. Evaluation Against Test Cases

All test cases executed successfully in the binary executable q2, matching expected calculations.

6. Screenshots of Compilation and Execution

```
GNU nano 9.0 q2.c *
#include <stdio.h>

int main(void) {
    double resistivity, length, area;
    printf("Enter resistivity (ohm-m), length (m), and cross-sectional area (m^2): ");
    if (scanf("%lf %lf %lf", &resistivity, &length, &area) != 3) {
        printf("Invalid Input\n");
        return 1;
    }

    if (resistivity <= 0 || length <= 0 || area <= 0) {
        printf("Invalid Input: Resistivity, length, and area must be positive.\n");
        return 1;
    }

    double resistance = (resistivity * length) / area;
    printf("Resistance: %.4f ohms\n", resistance);
    return 0;
}
```

```
(kali@kali)-[~/HDOC_Day7]
$ nano q2.c

(kali@kali)-[~/HDOC_Day7]
$ gcc q2.c -o q2
./q2
Enter resistivity (ohm-m), length (m), and cross-sectional area (m^2): 1.72e-8 100 0.000002
Resistance: 0.8600 ohms
```

Question 3: Mass Defect

1. Problem Statement Identification

- **Problem Statement:** Calculate the mass defect (Δm) of a nuclear reaction or system given its original mass and final mass.
- **Input:** Original mass (m_i), Final mass (m_f).
- **Output:** Mass defect (Δm).

• **Formula:**

$$\Delta m = m_i - m_f$$

2. Standard Test Case Table

Test Case	Input (original_mass, final_mass)	Expected Output	Actual Output	Result (Pass/Fail)
1	4.031882 4.001506	Mass Defect: 0.030376	Mass Defect: 0.030376	Pass
2	2.014102 2.013553	Mass Defect: 0.000549	Mass Defect: 0.000549	Pass
3	12.000000 11.996708	Mass Defect: 0.003292	Mass Defect: 0.003292	Pass
4	1.007825 1.007276	Mass Defect:	Mass Defect:	Pass

Test Case	Input (original_mass, final_mass)	Expected Output	Actual Output	Result (Pass/Fail)
		0.000549	0.000549	

3. Algorithm

1. Start

2. Read original_mass and final_mass.

3. If original_mass ≤ 0 or final_mass ≤ 0 , print "Invalid Input" and stop.

4. If final_mass $>$ original_mass, print "Invalid Input: Final mass cannot exceed original mass." and stop.

5. Calculate mass defect: mass_defect = original_mass - final_mass.

6. Print mass_defect.

7. End

4. C Program

```
#include <stdio.h>

int main(void) {
    double original_mass, final_mass;
    printf("Enter original mass (amu/kg) and final mass (amu/kg): ");
    if (scanf("%lf %lf", &original_mass, &final_mass) != 2) {
```

```
    printf("Invalid Input\n");
    return 1;
}

if (original_mass <= 0 || final_mass <= 0) {
    printf("Invalid Input: Masses must be positive.\n");
    return 1;
}

if (final_mass > original_mass) {
    printf("Invalid Input: Final mass cannot exceed original mass.\n");
    return 1;
}

double mass_defect = original_mass - final_mass;

printf("Mass Defect: %.6f\n", mass_defect);
return 0;
}
```

5. Evaluation Against Test Cases

All test cases executed successfully in the binary executable q3, matching expected calculations.

6. Screenshots of Compilation and Execution

```
Session Actions Edit View Help
GNU nano 9.0 q3.c *
#include <stdio.h>

int main(void) {
    double original_mass, final_mass;
    printf("Enter original mass (amu/kg) and final mass (amu/kg): ");
    if (scanf("%lf %lf", &original_mass, &final_mass) != 2) {
        printf("Invalid Input\n");
        return 1;
    }

    if (original_mass <= 0 || final_mass <= 0) {
        printf("Invalid Input: Masses must be positive.\n");
        return 1;
    }

    if (final_mass > original_mass) {
        printf("Invalid Input: Final mass cannot exceed original mass.\n");
        return 1;
    }

    double mass_defect = original_mass - final_mass;

    printf("Mass Defect: %.6f\n", mass_defect);
    return 0;
}
```

```
(kali㉿kali)-[~/HDOC_Day7]
$ nano q3.c

(kali㉿kali)-[~/HDOC_Day7]
$ gcc q3.c -o q3
./q3
Enter original mass (amu/kg) and final mass (amu/kg): 4.031882 4.001506
Mass Defect: 0.030376

(kali㉿kali)-[~/HDOC_Day7]
$
```